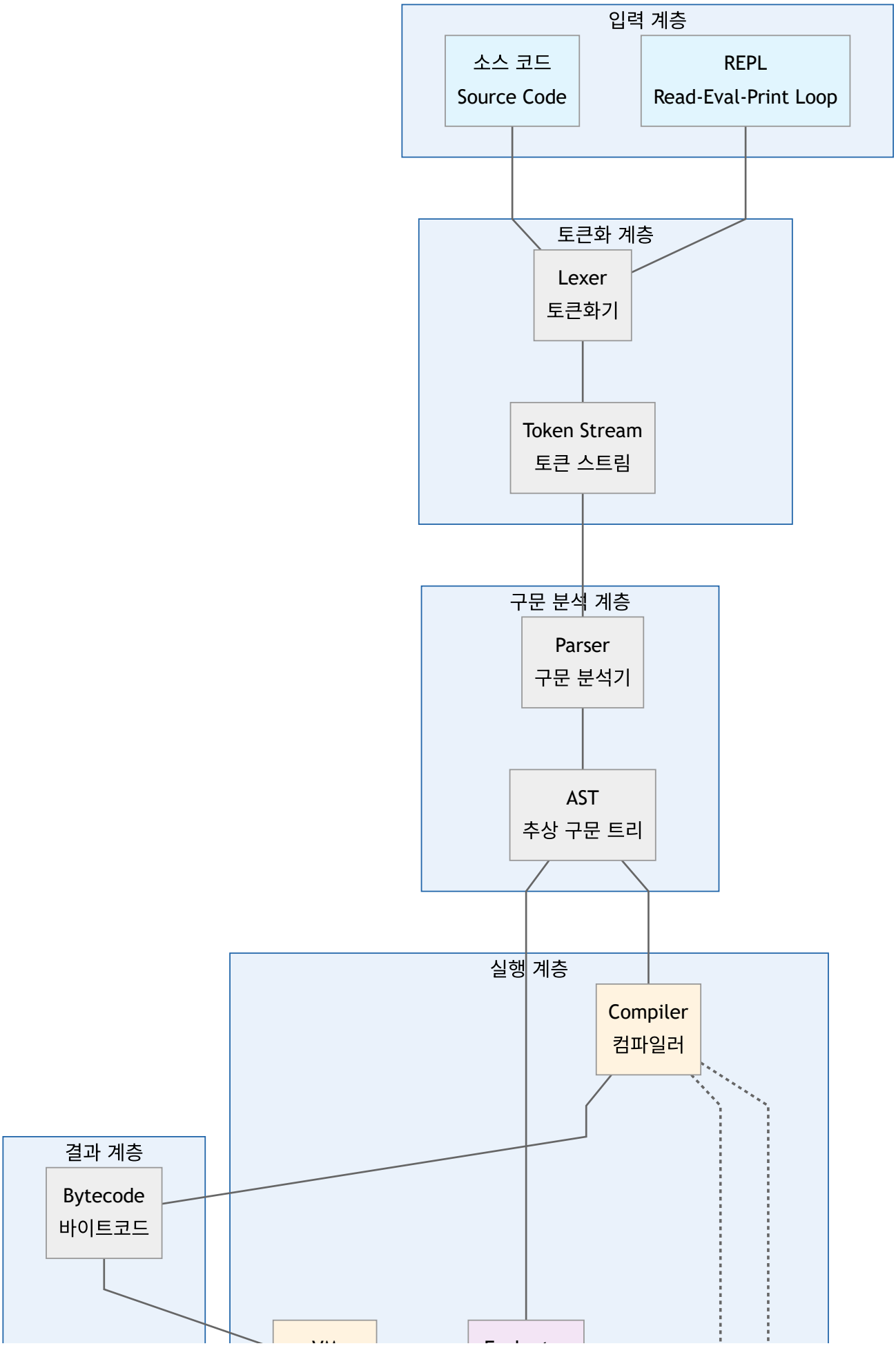
08. 전체 플로우 요약 (Complete Flow Summary)

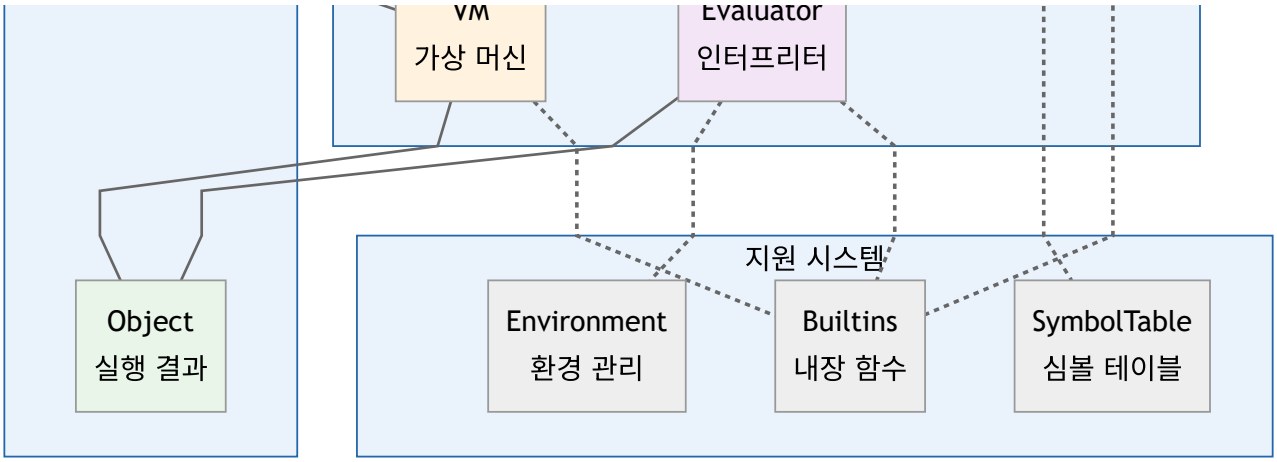
개요

이 문서는 Monkey 프로그래밍 언어의 인터프리터와 컴파일러 시스템의 전체적인 아키텍처와 데이터 흐름을 종합적으로 정리합니다. 각 컴포넌트 간의 관계, 실행 경로의 차이점, 그리고 핵심 학습 포인트들을 체계적으로 분석하여 시스템 전체에 대한 완전한 이해를 제공합니다.

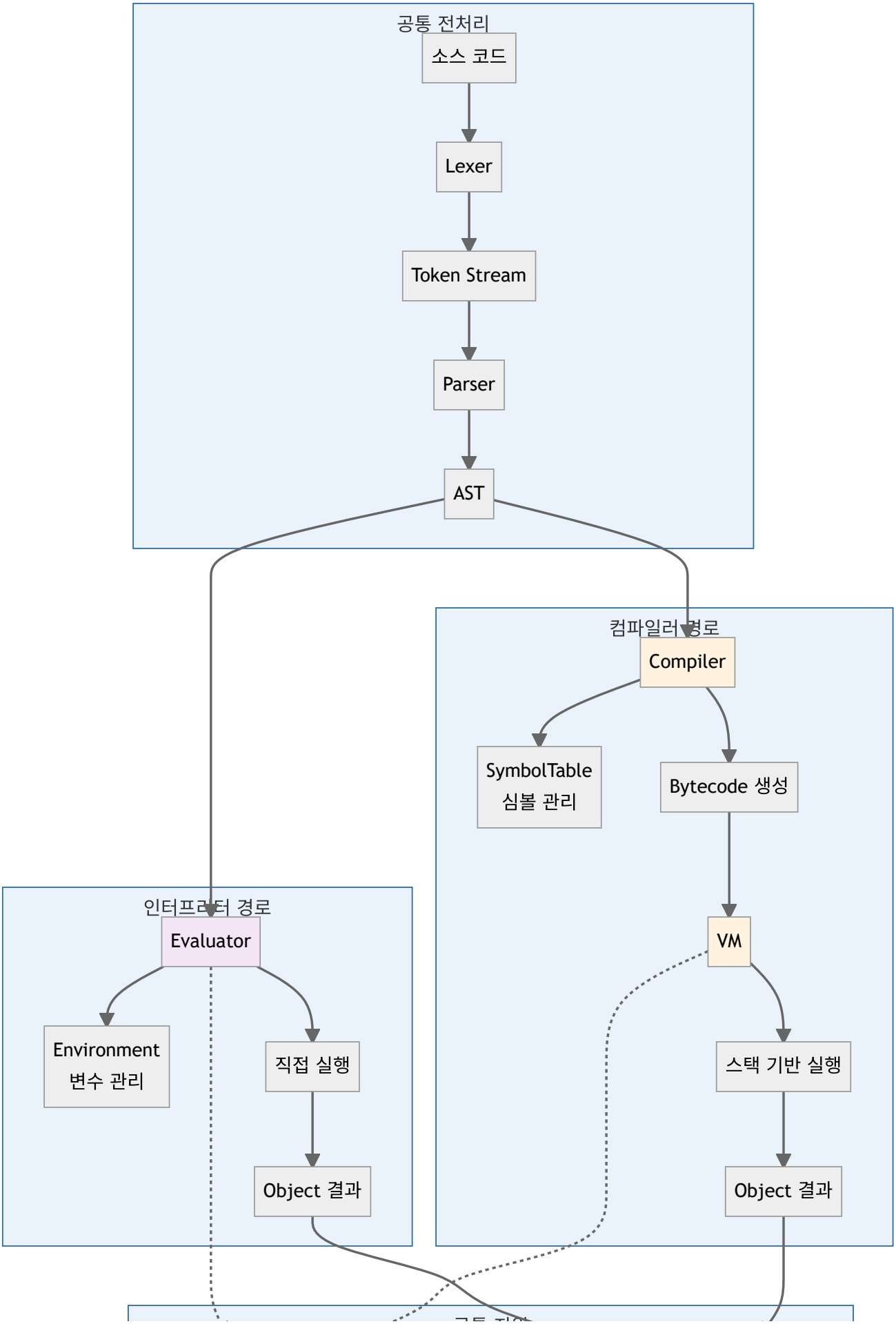
1. 전체 시스템 아키텍처 다이어그램

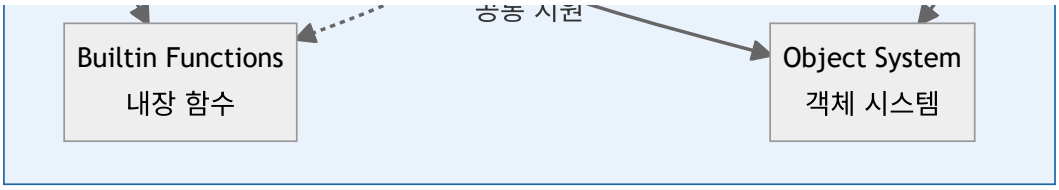
1.1 컴포넌트 간 관계도


입력 계층
소스 코드
Source Code
REPL
Read-Eval-Print Loop
토큰화 계층
Lexer
토큰화기
Token Stream
토큰 스트림
구문 분석 계층
Parser
구문 분석기
AST
추상 구문 트리
실행 계층
Compiler
컴파일러
결과 계층
Bytecode
바이트코드

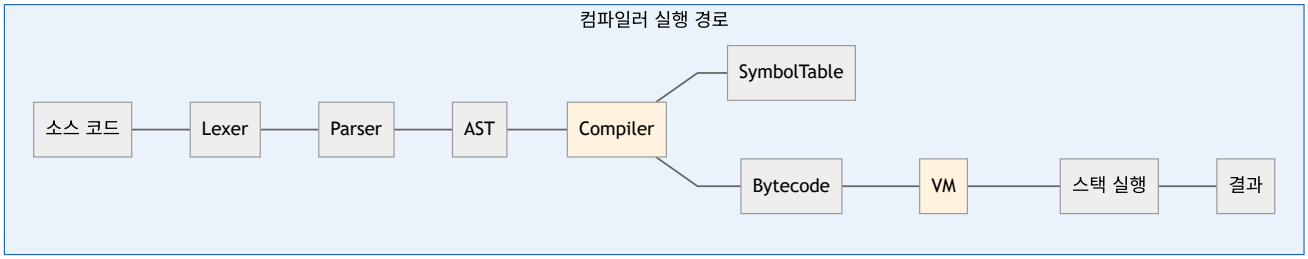

VM
가상 머신
Evaluator
인터프리터
Object
실행 결과
지원 시스템
Environment
환경 관리
Builtins
내장 함수
SymbolTable
심볼 테이블

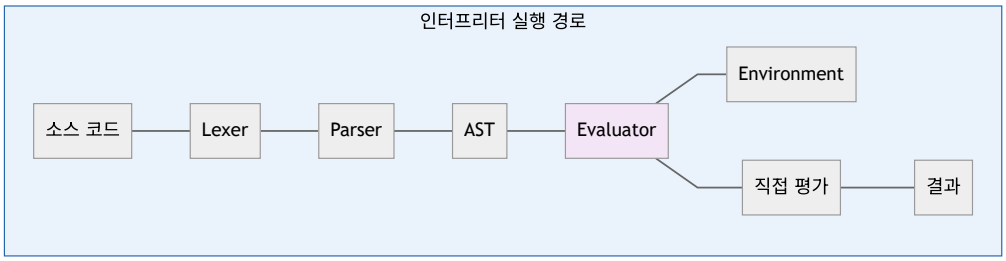
1.2 데이터 흐름도


공통 전처리
소스 코드
Lexer
Token Stream
Parser
AST
컴파일러 경로
Compiler
SymbolTable
심볼 관리
Bytecode 생성
VM
스택 기반 실행
Object 결과
인터프리터 경로
Evaluator
Environment
변수 관리
직접 실행
Object 결과



1.3 인터프리터와 컴파일러 실행 경로 비교





2. 핵심 개념과 학습 포인트 정리

2.1 각 단계별 핵심 개념 요약

토큰화 (Tokenization) 단계

핵심 개념:

- 소스 코드를 의미 있는 토큰 단위로 분해

- `Lexer` 구조체의 상태 관리 (position, nextPosition, ch)
- `NextToken()` 함수의 switch 문 기반 토큰 분류

주요 학습 포인트:

- 문자 단위 스캔과 토큰 생성 과정
- 키워드와 식별자 구분 메커니즘
- 다중 문자 연산자 처리 (`==` , `!=`)
- 문자열 리터럴과 이스케이프 처리

구문 분석 (Parsing) 단계

핵심 개념:

- Pratt Parser 알고리즘의 연산자 우선순위 처리
- AST 노드 시스템의 계층 구조 (Node → Statement/Expression)
- 전위/중위 파싱 함수 맵 기반 확장성

주요 학습 포인트:

- 재귀 하강 파싱과 연산자 우선순위
- AST 노드 설계와 `String()` 메서드 구현
- 에러 복구와 파싱 에러 수집
- 표현식과 문장의 구분

평가 (Evaluation) 단계 - 인터프리터

핵심 개념:

- AST 직접 순회를 통한 즉시 실행
- `Environment` 기반 스코프 체인 관리
- 객체 시스템을 통한 타입 안전성

주요 학습 포인트:

- 재귀적 AST 평가 과정
- 렉시컬 스코핑과 클로저 구현
- 함수 호출과 환경 확장
- 에러 전파와 처리 메커니즘

컴파일 (Compilation) 단계

핵심 개념:

- AST를 바이트코드로 변환하는 컴파일 과정
- `SymbolTable` 기반 심볼 관리와 스코프 처리
- 상수 풀과 명령어 생성 최적화

주요 학습 포인트:

- 바이트코드 명령어 설계와 피연산자 인코딩
- 스코프별 심볼 테이블 관리
- 점프 주소 패치와 제어 흐름
- 클로저와 자유 변수 처리

가상 머신 (VM) 실행 단계

핵심 개념:

- 스택 기반 바이트코드 실행 엔진
- `Frame` 시스템을 통한 함수 호출 관리
- 효율적인 명령어 디스패치

주요 학습 포인트:

- 스택 기반 연산의 원리와 장점

- 함수 호출 프레임과 지역 변수 관리
- 바이트코드 최적화 기법
- 메모리 관리와 가비지 컬렉션

시스템 통합 단계

핵심 개념:

- REPL을 통한 대화형 실행 환경
- 컴포넌트 간 데이터 흐름과 인터페이스
- 상태 관리와 세션 지속성

주요 학습 포인트:

- 전체 파이프라인의 데이터 변환 과정
- 에러 전파와 사용자 피드백
- 성능 특성과 메모리 사용 패턴
- 확장성과 모듈화 설계

2.2 구현 시 주의사항과 베스트 프랙티스

메모리 관리

```
// 좋은 예: 효율적인 슬라이스 관리
func (l *Lexer) readIdentifier() string {
position := l.position
for  isLetter(l.ch) {
l.readChar()
}
```

```go
return l.input[position:l.position] // 새로운 문자열 할당 최소화
}

// 나쁜 예: 불필요한 문자열 연결
func (l *Lexer) readIdentifierBad() string {
result := ""
for isLetter(l.ch) {
result += string(l.ch) // 매번 새로운 문자열 생성
l.readChar()
}
return result
}
```

에러 처리

```go
// 좋은 예: 명확한 에러 메시지와 위치 정보
func (p *Parser) parseLetStatement() *ast.LetStatement {
if !p.expectPeek(token.Identifier) {
p.errors = append(p.errors,
fmt.Sprintf("expected next token to be %s, got %s instead",
token.Identifier, p.peekToken.Type))
return nil
}
// ...
```

```go
}

// 나쁜 예: 모호한 에러 메시지
func (p *Parser) parseLetStatementBad() *ast.LetStatement {
if !p.expectPeek(token.Identifier) {
p.errors = append(p.errors, "parsing error") // 정보 부족
return nil
}
// ...
}
```

성능 최적화

```go
// 좋은 예: 미리 계산된 우선순위 맵
var precedences = map[token.TokenType]int{
token.Equal: Equals,
token.NotEqual: Equals,
token.LessThan: LessOrGreater,
// ...
}

func (p *Parser) peekPrecedence() int {
if p, ok := precedences[p.peekToken.Type]; ok {
```

```go
return p
}
return Lowest
}

// 나쁜 예: 매번 계산하는 우선순위
func (p *Parser) peekPrecedenceBad() int {
switch p.peekToken.Type {
case token.Equal, token.NotEqual:
return Equals
case token.LessThan, token.GreaterThan:
return LessOrGreater
// ... 매번 switch 문 실행
}
}
```

확장성 고려

```go
// 좋은 예: 인터페이스 기반 확장성
type Node interface {
TokenLiteral() string
String() string
}
```

```go
type Statement interface {
Node
statementNode()
}

// 새로운 문장 타입 추가가 용이
type WhileStatement struct {
Token token.Token
Condition Expression
Body *BlockStatement
}

func (ws *WhileStatement) statementNode() {}
func (ws *WhileStatement) TokenLiteral() string { return  ws.Token.Literal }
func (ws *WhileStatement) String() string { /* 구현 */ }
```

테스트 주도 개발

```go
// 좋은 예: 포괄적인 테스트 케이스
func  TestLetStatements(t *testing.T) {
tests := []struct {
input  string
expectedIdentifier  string
```

```go
expectedValue interface{}
}{
{"let x = 5;", "x", 5},
{"let y = true;", "y", true},
{"let foobar = y;", "foobar", "y"},
}

for _, tt := range tests {
l := lexer.New(tt.input)
p := New(l)
program := p.ParseProgram()
checkParserErrors(t, p)
// 상세한 검증 로직
}
}
```

2.3 디버깅과 문제 해결 전략

AST 시각화

```go
// AST 구조를 시각적으로 확인하는 헬퍼 함수
func printAST(node ast.Node, indent int) {
spaces := strings.Repeat(" ", indent)
switch n := node.(type) {
```

```go
case *ast.Program:
fmt.Printf("%sProgram:\n", spaces)
for _, stmt := range n.Statements {
printAST(stmt, indent+1)
}
case *ast.LetStatement:
fmt.Printf("%sLetStatement: %s = \n", spaces, n.Name.Value)
printAST(n.Value, indent+1)
// ... 다른 노드 타입들
}
}
```

바이트코드 디스어셈블리

```go
// 바이트코드를 사람이 읽을 수 있는 형태로 출력
func disassemble(instructions code.Instructions) {
fmt.Println("=== Disassembly ===")
i := 0
for i < len(instructions) {
def, err := code.Lookup(instructions[i])
if err != nil {
fmt.Printf("ERROR: %s\n", err)
continue
}
```

```
operands, read := code.ReadOperands(def, instructions[i+1:])
fmt.Printf("%04d %s %v\n", i, def.Name, operands)
i += 1 + read
}
}
```

실행 추적

```
// VM 실행 과정을 단계별로 추적
func (vm *VM) RunWithTrace() error {
for vm.currentFrame().ip < len(vm.currentFrame().Instructions())-1 {
vm.currentFrame().ip++
ip := vm.currentFrame().ip
ins := vm.currentFrame().Instructions()
op := code.Opcode(ins[ip])
fmt.Printf("IP: %04d, OP: %s, Stack: %v\n",
ip, code.OpcodeName(op), vm.stackTop())
// 명령어 실행
switch op {
// ... 각 명령어 처리
}
}
return nil
```

}

이러한 핵심 개념들과 베스트 프랙티스를 이해하고 적용함으로써, Monkey 언어 시스템의 각 컴포넌트를 효과적으로 구현하고 확장할 수 있습니다.

---

3. 추가 학습 자료와 확장 방향 제시

3.1 더 깊이 있는 학습을 위한 자료 추천

컴파일러 이론 및 구현

추천 도서:

- "Crafting Interpreters" by Robert Nystrom
- 트리 워킹 인터프리터와 바이트코드 VM 구현
- 실용적인 언어 구현 기법
- "Engineering a Compiler" by Keith Cooper & Linda Torczon
- 컴파일러 최적화 기법
- 고급 파싱 알고리즘
- "Modern Compiler Implementation" by Andrew Appel
- 함수형 언어를 사용한 컴파일러 구현
- 타입 시스템과 최적화

온라인 리소스:

- LLVM Tutorial: https://llvm.org/docs/tutorial/
- Pratt Parser 원본 논문: "Top Down Operator Precedence" by Vaughan Pratt
- Stanford CS143 (Compilers): 온라인 강의 자료

가상 머신과 바이트코드 설계

학습 주제:

- JVM 바이트코드 명세서 분석
- Python 바이트코드 (CPython) 구조 연구
- WebAssembly (WASM) 설계 원리
- Lua VM 구현 분석

프로그래밍 언어 설계

핵심 개념:

- 타입 시스템 설계 (정적 vs 동적)
- 메모리 관리 전략 (GC, 참조 카운팅)
- 동시성 모델 (Actor, CSP, Async/Await)
- 모듈 시스템과 네임스페이스

3.2 언어 기능 확장 아이디어와 구현 방향

3.2.1 타입 시스템 추가

정적 타입 검사 구현:

```
// 타입 정보를 포함하는 AST 노드
type TypedIdentifier struct {
Name  string
Type TypeInfo
}
```

```
type TypeInfo struct {
Kind TypeKind // Int, String, Function, etc.
Params []TypeInfo // 함수 매개변수 타입
Return TypeInfo // 함수 반환 타입
}

// 타입 검사기 구현
type TypeChecker struct {
typeEnv  map[string]TypeInfo
errors []string
}

func (tc *TypeChecker) CheckProgram(program *ast.Program) []string {
for  _, stmt := range  program.Statements {
tc.checkStatement(stmt)
}
return  tc.errors
}
```

제네릭 타입 지원:

```
// 목표 문법
fn map<T, U>(arr: Array<T>, fn: Function<T, U>) -> Array<U> {
let result = [];
```

```
for (item in arr) {
result = push(result, fn(item));
}
return result;
}

let numbers = [1, 2, 3];
let strings = map(numbers, fn(x) { str(x) });
```

3.2.2 고급 제어 구조

반복문 추가:

```
// for 루프
for (let i = 0; i < 10; i = i + 1) {
puts(i);
}

// for-in 루프 (이터레이터)
for (item in [1, 2, 3]) {
puts(item);
}

// while 루프
```

```
while (condition) {
// 본문
}
```

패턴 매칭:

```
// match 표현식
let result = match value {
0 => "zero",
1 | 2 => "one or two",
n if n > 10 => "big number",
_ => "other"
};

// 구조 분해
let {name, age} = person;
let [first, ...rest] = array;
```

3.2.3 모듈 시스템

모듈 정의와 임포트:

```
// math.monkey
export fn add(a, b) { a + b }
export fn multiply(a, b) { a * b }
```

```
export const PI = 3.14159;


// main.monkey
import {add, PI} from "./math";
import * as math from "./math";


let result = add(1, 2);
let area = PI * radius * radius;
```

모듈 해결 시스템:

```
type ModuleResolver struct {
cache  map[string]*ast.Program
basePath  string
}


func (mr *ModuleResolver) ResolveModule(path  string) (*ast.Program, error) {
if  cached, ok := mr.cache[path];  ok {
return  cached, nil
}
fullPath := filepath.Join(mr.basePath, path+".monkey")
source, err := ioutil.ReadFile(fullPath)
if  err != nil {
return  nil, err
```

```
}

// 파싱 및 캐싱

program := parseModule(string(source))

mr.cache[path] = program

return  program, nil

}
```

3.2.4 동시성 지원

고루틴 스타일 동시성:

```
// 채널과 고루틴

let ch = make_channel();


go fn() {

send(ch, "Hello from goroutine!");

};


let message = receive(ch);

puts(message);
```

Actor 모델:

```
// Actor 정의

let counter_actor = actor {
```

```
let count = 0;
receive {
"increment" => {
count = count + 1;
reply(count);
},
"get" => reply(count)
}
};

// Actor 사용
let counter = spawn(counter_actor);
send(counter, "increment");
let result = receive_reply();
```

3.2.5 메모리 관리 개선

가비지 컬렉터 구현:

```
type GarbageCollector struct {
heap []Object
roots []Object
markBits []bool
}
```

```go
func (gc *GarbageCollector) Collect() {
// Mark phase
gc.markReachable()
// Sweep phase
gc.sweepUnreachable()
// Compact phase (optional)
gc.compactHeap()
}

func (gc *GarbageCollector) markReachable() {
for _, root := range gc.roots {
gc.mark(root)
}
}
```

참조 카운팅 최적화:

```go
type RefCountedObject struct {
Object
refCount int32
}

func (obj *RefCountedObject) Retain() {
```

```
atomic.AddInt32(&obj.refCount, 1)
}

func (obj *RefCountedObject) Release() {
if  atomic.AddInt32(&obj.refCount, -1) == 0 {
obj.deallocate()
}
}
```

3.2.6 성능 최적화

JIT 컴파일러 구현:

```
type JITCompiler struct {
hotspots  map[*ast.Function]int
threshold  int
nativeCode  map[*ast.Function]func([]Object) Object
}

func (jit *JITCompiler) ShouldCompile(fn *ast.Function) bool {
count := jit.hotspots[fn]
return  count > jit.threshold
}
```

```go
func (jit *JITCompiler) CompileToNative(fn *ast.Function) {
// LLVM IR 생성 또는 어셈블리 코드 생성
nativeFunc := jit.generateNativeCode(fn)
jit.nativeCode[fn] = nativeFunc
}
```

인라인 캐싱:

```go
type InlineCache struct {
receiverType ObjectType
method  func(Object, []Object) Object
hitCount  int
}

func (ic *InlineCache) Call(receiver Object, args []Object) Object {
if  receiver.Type() == ic.receiverType {
ic.hitCount++
return  ic.method(receiver, args)
}
// 캐시 미스 - 새로운 메서드 조회 및 캐싱
method := lookupMethod(receiver.Type(), methodName)
ic.receiverType = receiver.Type()
ic.method = method
ic.hitCount = 1
```

```
return  method(receiver, args)
}
```

3.3 실제 프로덕션 언어로의 발전 방향

표준 라이브러리 구축

```
// 파일 I/O
import "io"
let file = io.open("data.txt", "r");
let content = io.read_all(file);
io.close(file);

// HTTP 클라이언트
import "http"
let response = http.get("https://api.example.com/data");
let json_data = json.parse(response.body);

// 정규 표현식
import "regex"
let pattern = regex.compile(r"\d+");
let matches = pattern.find_all("abc123def456");
```

패키지 관리 시스템

```
# 패키지 설치
monkey install github.com/user/awesome-lib

# 의존성 관리
monkey.toml:
[dependencies]
awesome-lib = "1.2.3"
http-client = "^2.0.0"
```

개발 도구 생태계

- **LSP (Language Server Protocol)** 구현
- **디버거** 개발 (DAP 프로토콜 지원)
- **포매터** 및 **린터** 도구
- **테스트 프레임워크** 구축
- **벤치마킹** 도구 개발

크로스 플랫폼 지원

```
// 플랫폼별 컴파일 타겟
type CompileTarget struct {
OS string // windows, linux, darwin
Arch string // amd64, arm64, wasm
}
```

```go
func (c *Compiler) CompileForTarget(target CompileTarget) (*Bytecode, error) {
// 플랫폼별 최적화 적용
switch target.OS {
case "wasm":
return c.compileToWasm()
case "native":
return c.compileToNative(target.Arch)
default:
return c.compileToBytecode()
}
}
```

3.4 학습 로드맵 제안

초급 단계 (1-2개월)

1. **기본 구조 이해**

- 각 컴포넌트의 역할과 상호작용
- 토큰화와 파싱 과정 실습
- 간단한 AST 노드 추가

2. **실습 프로젝트**

- 새로운 연산자 추가 (`%`, `**`)
- 새로운 리터럴 타입 추가 (float)
- 간단한 내장 함수 구현

중급 단계 (2-3개월)

1. **고급 기능 구현**

- 반복문 구조 추가
- 예외 처리 메커니즘
- 기본적인 타입 검사

2. **성능 최적화**

- 바이트코드 최적화 기법
- 메모리 사용량 분석
- 벤치마킹과 프로파일링

고급 단계 (3-6개월)

1. **언어 설계**

- 완전한 타입 시스템 설계
- 모듈 시스템 구현
- 동시성 모델 추가

2. **도구 개발**

- 디버거 구현
- IDE 플러그인 개발
- 패키지 관리자 구축

이러한 확장 방향들을 통해 Monkey 언어를 실용적인 프로그래밍 언어로 발전시킬 수 있으며, 각 단계별 학습을 통해 프로그래밍 언어 구현에 대한 깊이 있는 이해를 얻을 수 있습니다.

결론

Monkey 프로그래밍 언어의 인터프리터와 컴파일러 시스템은 현대적인 언어 구현의 핵심 개념들을 모두 포함하고 있는 완성도 높은 교육용 프로젝트입니다.

주요 성취:

- 완전한 렉서-파서-평가기 파이프라인 구현
- 스택 기반 가상 머신과 바이트코드 컴파일러 구현
- 클로저와 고차 함수를 지원하는 함수형 프로그래밍 기능
- 확장 가능한 내장 함수 시스템
- 두 가지 실행 방식의 성능 특성 비교

학습 가치:

- 프로그래밍 언어 구현의 전체 과정 이해
- 컴파일러 이론의 실제 적용
- 시스템 프로그래밍과 성능 최적화 기법
- 소프트웨어 아키텍처 설계 원칙

이 시스템을 기반으로 더 복잡하고 실용적인 프로그래밍 언어를 구현할 수 있는 견고한 기반을 마련할 수 있습니다. 각 컴포넌트의 설계 원칙과 구현 기법들은 다른 언어 프로젝트나 컴파일러 개발에도 직접 적용할 수 있는 가치 있는 지식입니다.